WinForms VB.NET MVP Application Guide

Section 1: Introduction to MVP and SQL Schema

The Model-View-Presenter (MVP) pattern is used to separate application logic from the user interface.

SQL Database Tables

Here are the SQL table definitions for the Customers and Invoices tables.

Customers Table:

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Email NVARCHAR(100),
    Phone NVARCHAR(20)
);
```

Invoices Table:

```
CREATE TABLE Invoices (
    InvoiceID INT PRIMARY KEY IDENTITY(1,1),
    InvoiceDate DATE,
    CustomerID INT,
    TotalAmount DECIMAL(18, 2),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Section 2: Core MVP Components

Data Transfer Objects (DTOs)

These are simple classes to transfer data between layers.

CustomerDTO.vb

```
Public Class CustomerDTO
```

```
    Public Property CustomerID As Integer
    Public Property FirstName As String
    Public Property LastName As String
    Public Property Email As String
End Class
```

InvoiceDTO.vb

```
Public Class InvoiceDTO
    Public Property InvoiceID As Integer
    Public Property InvoiceDate As Date
    Public Property CustomerID As Integer
    Public Property TotalAmount As Decimal
End Class
```

Model Layer (Data Access)

The repository handles all database interactions.

CustomerRepository.vb

```
Imports System.Data.SqlClient

Public Class CustomerRepository
    Private ReadOnly _connectionString As String

    Public Sub New(connectionString As String)
        _connectionString = connectionString
    End Sub

    Public Function GetCustomers() As List(Of CustomerDTO)
        Dim customers As New List(Of CustomerDTO)()
        Using connection As New SqlConnection(_connectionString)
            Dim sql As String = "SELECT CustomerID, FirstName, LastName, Email FROM
Customers"
            Using command As New SqlCommand(sql, connection)
                connection.Open()
                Using reader As SqlDataReader = command.ExecuteReader()
                    While reader.Read()
                        Dim customer As New CustomerDTO()
                        customer.CustomerID = reader.GetInt32(reader.GetOrdinal("CustomerID"))
                        customer.FirstName = reader.GetString(reader.GetOrdinal("FirstName"))
```

```
                customer.LastName = reader.GetString(reader.GetOrdinal("LastName"))
                customer.Email = If(reader.IsDBNull(reader.GetOrdinal("Email")), Nothing,
reader.GetString(reader.GetOrdinal("Email")))
                customers.Add(customer)
            End While
        End Using
    End Using
    End Using
    Return customers
End Function

' Add other methods like AddCustomer, UpdateCustomer, DeleteCustomer
End Class
```

View Layer (Interface and Form)

The View is the UI and defines an interface for the Presenter.

ICustomerView.vb

```
Public Interface ICustomerView
    Event LoadView As EventHandler
    Event SaveCustomer(customer As CustomerDTO)

    Sub DisplayCustomers(customers As List(Of CustomerDTO))
    Sub ShowMessage(message As String)
End Interface
```

FrmMain.vb (Partial Class)

```
Public Partial Class FrmMain
    Implements ICustomerView

    Private _presenter As CustomerPresenter

    Public Sub New()
        InitializeComponent()
        Dim connectionString As String = "Your connection string here"
        Dim customerModel As New CustomerRepository(connectionString)
        _presenter = New CustomerPresenter(Me, customerModel)
    End Sub
```

```
    Public Sub DisplayCustomers(customers As List(Of CustomerDTO)) Implements
ICustomerView.DisplayCustomers
        Me.dgvCustomers.DataSource = customers
    End Sub

    Public Sub ShowMessage(message As String) Implements ICustomerView.ShowMessage
        MessageBox.Show(message)
    End Sub

    Private Sub FrmMain_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        RaiseEvent LoadView(Me, EventArgs.Empty)
    End Sub

    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles btnSave.Click
        Dim newCustomer As New CustomerDTO()
        newCustomer.FirstName = Me.txtFirstName.Text
        newCustomer.LastName = Me.txtLastName.Text
        newCustomer.Email = Me.txtEmail.Text
        RaiseEvent SaveCustomer(newCustomer)
    End Sub
End Class
```

Presenter Layer

The Presenter contains the business logic.

CustomerPresenter.vb

```
Public Class CustomerPresenter
    Private ReadOnly _view As ICustomerView
    Private ReadOnly _model As CustomerRepository

    Public Sub New(view As ICustomerView, model As CustomerRepository)
        _view = view
        _model = model
        AddHandler _view.LoadView, AddressOf OnLoadView
        AddHandler _view.SaveCustomer, AddressOf OnSaveCustomer
    End Sub

    Private Sub OnLoadView(sender As Object, e As EventArgs)
        Try
            Dim customers As List(Of CustomerDTO) = _model.GetCustomers()
```

```
            _view.DisplayCustomers(customers)
        Catch ex As Exception
            _view.ShowMessage("Error loading customers: " & ex.Message)
        End Try
    End Sub

    Private Sub OnSaveCustomer(sender As Object, e As CustomerDTO)
        Dim result As ValidationResult = _model.AddCustomer(e)
        If result.IsValid Then
            _view.ShowMessage("Customer saved successfully.")
            OnLoadView(Nothing, EventArgs.Empty)
        Else
            _view.ShowMessage("Validation Error: " & result.ErrorMessage)
        End If
    End Sub
End Class
```

Section 3: Adding Validation and Reusability

Validation Result Class

A simple class to return validation outcomes.

ValidationResult.vb

```
Public Class ValidationResult
    Public Property IsValid As Boolean
    Public Property ErrorMessage As String

    Public Shared Function Success() As ValidationResult
        Return New ValidationResult() With {.IsValid = True}
    End Function

    Public Shared Function Fail(message As String) As ValidationResult
        Return New ValidationResult() With {.IsValid = False, .ErrorMessage = message}
    End Function
End Class
```

Generic Validation Utility

A reusable class for common validation rules.

ValidationUtils.vb

```
Imports System.Text.RegularExpressions

Public NotInheritable Class ValidationUtils
    Private Sub New()
    End Sub

    Public Shared Function IsNotNullOrEmpty(value As String) As Boolean
        Return Not String.IsNullOrWhiteSpace(value)
    End Function

    Public Shared Function IsValidEmail(email As String) As Boolean
        If String.IsNullOrWhiteSpace(email) Then
            Return False
        End If
        Dim pattern As String = "^[^@\s]+@[^@\s]+\.[^@\s]+$"
        Dim regex As New Regex(pattern, RegexOptions.IgnoreCase)
        Return regex.IsMatch(email)
    End Function
End Class
```

Section 4: Optimal Design and Best Practices

Dependency Injection (DI) and Async/Await

Using a Program.vb entry point to manage dependencies and asynchronous operations.

Program.vb

```
Imports System.Windows.Forms

Module Program
    <STAThread>
    Sub Main()
        Dim connectionString As String = "Your connection string here"
        Dim customerModel As New CustomerRepository(connectionString)
        Dim customerView As New FrmMain()
        Dim customerPresenter As New CustomerPresenter(customerView, customerModel)
        Application.Run(customerView)
    End Sub
End Module
```

Messaging Service

A dedicated service for displaying messages.

IMessagingService.vb

```
Public Interface IMessagingService
    Sub ShowSuccess(message As String)
    Sub ShowError(message As String)
    Sub ShowInformation(message As String)
End Interface
```

WinFormsMessageBoxService.vb

```
Imports System.Windows.Forms

Public Class WinFormsMessageBoxService
    Implements IMessagingService

    Public Sub ShowSuccess(message As String) Implements IMessagingService.ShowSuccess
        MessageBox.Show(message, "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information)
    End Sub

    Public Sub ShowError(message As String) Implements IMessagingService.ShowError
        MessageBox.Show(message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error)
    End Sub

    Public Sub ShowInformation(message As String) Implements
IMessagingService.ShowInformation
        MessageBox.Show(message, "Information", MessageBoxButtons.OK,
MessageBoxIcon.Information)
    End Sub
End Class
```

StatusBarMessagingService.vb

```
Imports System.Windows.Forms

Public Class StatusBarMessagingService
```

```
    Implements IMessagingService

    Private ReadOnly _statusLabel As ToolStripStatusLabel
    Private ReadOnly _statusStrip As StatusStrip

    Public Sub New(statusStrip As StatusStrip)
        If statusStrip Is Nothing Then
            Throw New ArgumentNullException(NameOf(statusStrip), "StatusStrip cannot be null.")
        End If
        If statusStrip.Items.Count = 0 OrElse Not (TypeOf statusStrip.Items(0) Is
ToolStripStatusLabel) Then
            Throw New ArgumentException("The StatusStrip must contain at least one
ToolStripStatusLabel.", NameOf(statusStrip))
        End If

        _statusStrip = statusStrip
        _statusLabel = CType(statusStrip.Items(0), ToolStripStatusLabel)
    End Sub

    Public Sub ShowSuccess(message As String) Implements IMessagingService.ShowSuccess
        _statusStrip.BeginInvoke(Sub()
                                     _statusLabel.Text = "Success: " & message
                                     _statusLabel.ForeColor = Color.Green
                                 End Sub)
    End Sub

    Public Sub ShowError(message As String) Implements IMessagingService.ShowError
        _statusStrip.BeginInvoke(Sub()
                                     _statusLabel.Text = "Error: " & message
                                     _statusLabel.ForeColor = Color.Red
                                 End Sub)
    End Sub

    Public Sub ShowInformation(message As String) Implements
IMessagingService.ShowInformation
        _statusStrip.BeginInvoke(Sub()
                                     _statusLabel.Text = "Info: " & message
                                     _statusLabel.ForeColor = Color.Black
                                 End Sub)
    End Sub
End Class
```

Section 5: Modular Architecture

To scale the application, modules are used to register services with a Dependency Injection container.

CustomerModule.vb

```
Imports System.Collections.Generic

Public Class CustomerModule
    Private ReadOnly _container As Dictionary(Of Type, Object)

    Public Sub New(container As Dictionary(Of Type, Object))
        _container = container
    End Sub

    Public Sub RegisterServices()
        _container.Add(GetType(CustomerRepository), New
CustomerRepository("YourConnectionString"))
        _container.Add(GetType(CustomerPresenter), New
CustomerPresenter(CType(_container(GetType(FrmMain)), ICustomerView),
CType(_container(GetType(CustomerRepository)), CustomerRepository),
CType(_container(GetType(IMessagingService)), IMessagingService)))
    End Sub
End Class
```